Chapter 8: Fractional Factorial Designs

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Section 8.1: Two-level Designs **1**
2.1 Half-normal Plot 1

3 Section 8.2: Bayesian-inspired Designs **2**
3.1 Bayesian Half-normal Plot with HiGarrote 2
3.2 HiGarrote Variable Selection 4

4 Section 8.3: Designs with More than Two Levels **7**
4.1 Orthogonal Arrays (L9, L18) 7

5 Section 8.4: Mixture Designs **7**
5.1 Simplex-Lattice and Simplex-Centroid Designs 7
5.2 Support Points on Simplex 8

6 Summary **9**

1 Introduction

This chapter covers fractional factorial designs, Bayesian-inspired designs, and mixture designs.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering*. CRC Press.

```
library(rkriging)
library(HiGarrote)
```

2 Section 8.1: Two-level Designs

2.1 Half-normal Plot

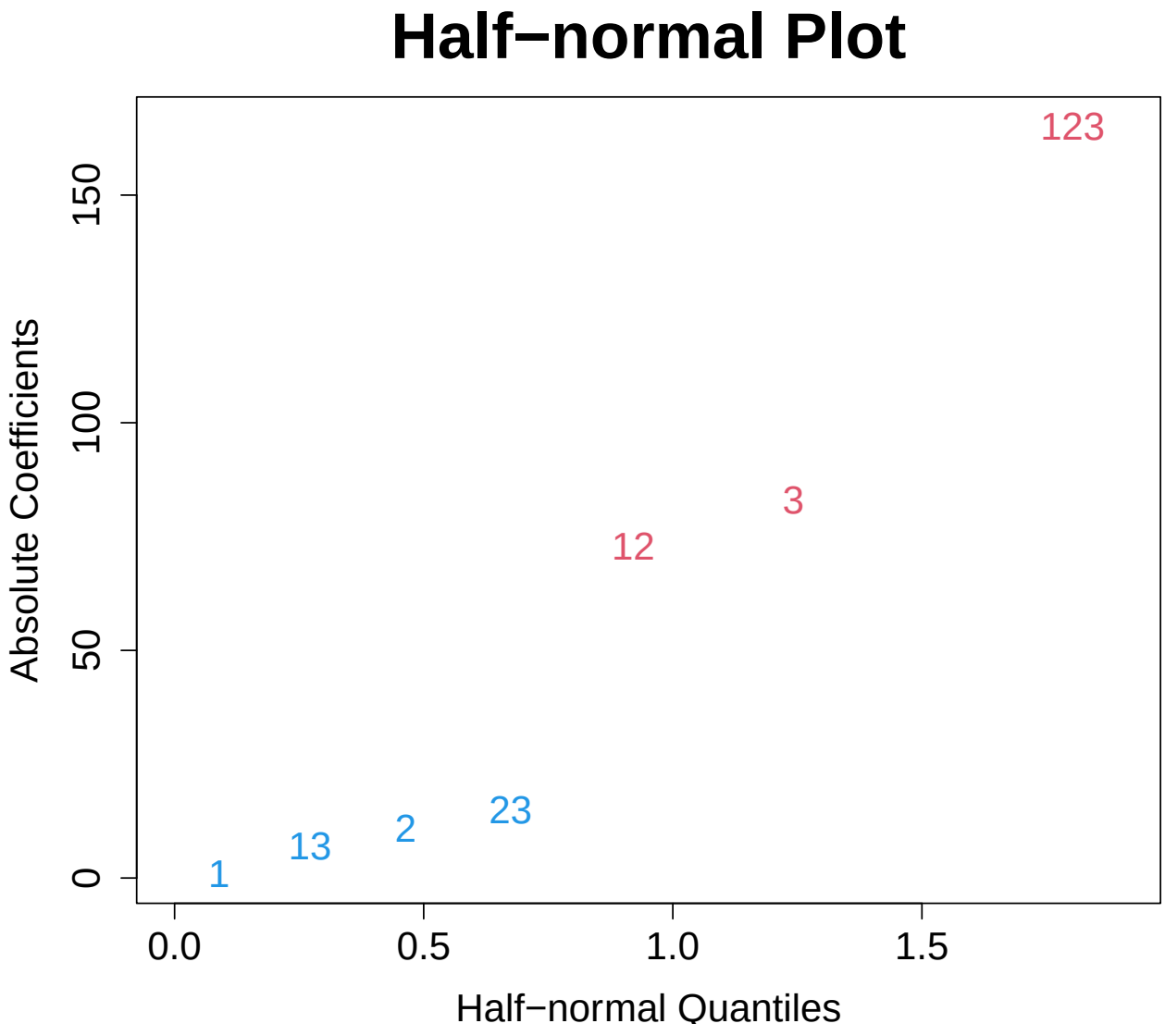
```
A <- rep(c(-1, 1), c(4, 4))
B <- rep(rep(c(-1, 1), c(2, 2)), 2)
C <- rep(c(-1, 1), 4)
D <- A * B * C

y <- c(504, 984, 928, 808, 992, 784, 464, 976)
a <- lm(y ~ (A + B + C)^3)

eff <- a$coef[-1]
```

```r
names(eff) <- c("1", "2", "3", "12", "13", "23", "123")
I <- length(eff)
u <- qnorm(0.5 + 0.5 * (1:I - 0.5) / I)
saeff <- sort(abs(eff))

par(mar = c(5, 5, 4, 2))
plot(u, saeff, type = "n", xlab = "Half-normal Quantiles",
     ylab = "Absolute Coefficients", main = "Half-normal Plot",
     cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5, xlim = c(0, max(u) + 0.1))
text(u, saeff, names(saeff), cex = 1.5, col = c(rep(4, 4), rep(2, 3)))
```



3 Section 8.2: Bayesian-inspired Designs

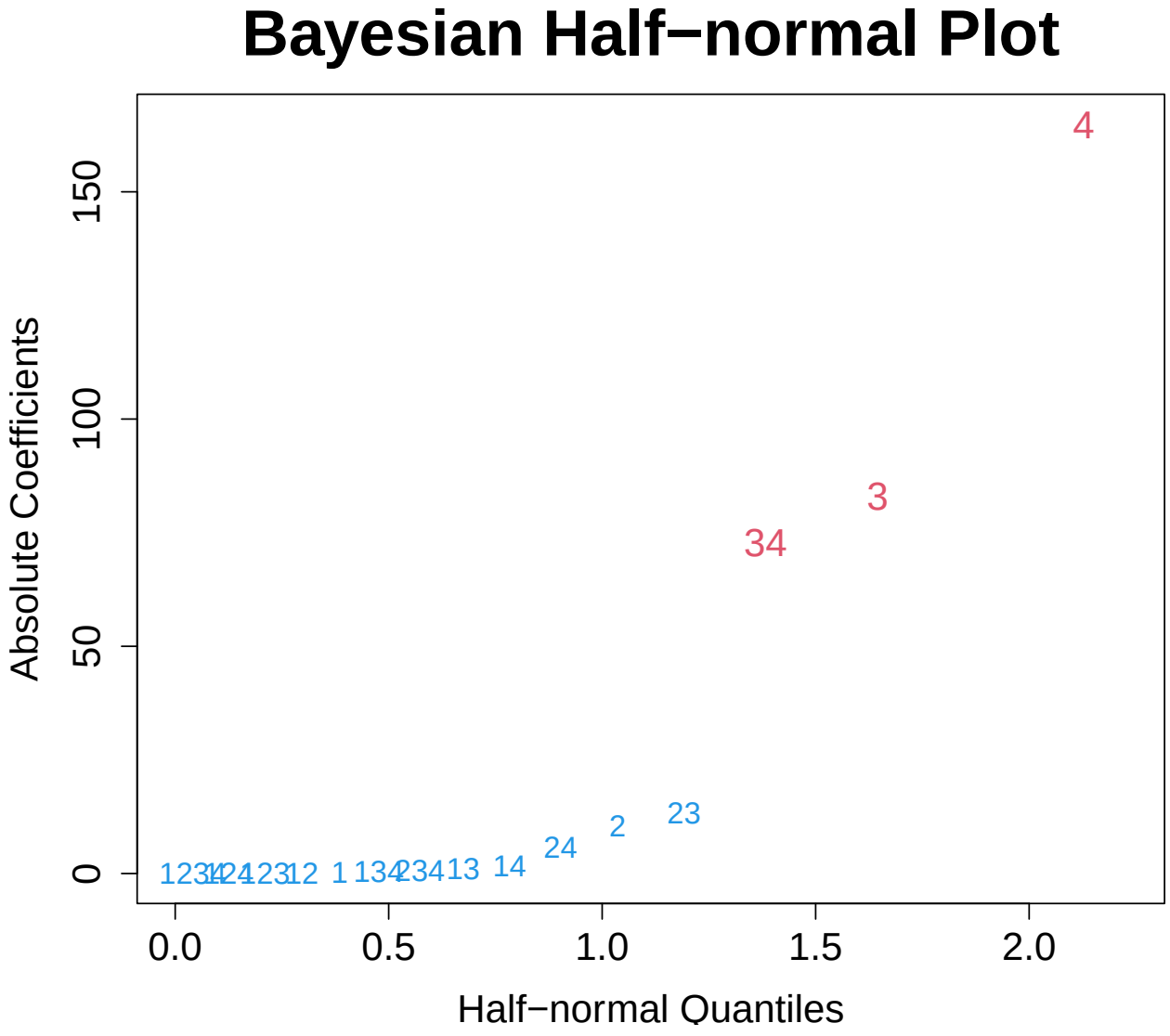
3.1 Bayesian Half-normal Plot with HiGarrote

```r
d <- cbind(A, B, C, D)
a <- Fit.Kriging(d, y, kernel.parameters = list(type = "Gaussian"))
```

```
theta <- Get.Kriging.Parameters(a)$lengthscale
mu <- Get.Kriging.Parameters(a)$mu
gam <- (1 - exp(-2^2 / (2 * theta))) / (1 + exp(-2^2 / (2 * theta)))

X <- model.matrix(lm(y ~ (A + B + C + D)^4), data = data.frame(A, B, C, D))
G <- diag(c(model.matrix(lm(1 ~ (gam[1] + gam[2] + gam[3] + gam[4])^4))))
alpha <- G %*% t(X) %*% solve(X %*% G %*% t(X)) %*% (y - mu)
eff <- alpha[-1]
names(eff) <- c("1", "2", "3", "4", "12", "13", "14", "23", "24", "34",
                "123", "124", "134", "234", "1234")
I <- length(eff)
u <- qnorm(0.5 + 0.5 * (1:I - 0.5) / I)
saeff <- sort(abs(eff))

par(mar = c(5, 5, 4, 2))
plot(u, saeff, type = "n", xlab = "Half-normal Quantiles",
     ylab = "Absolute Coefficients", main = "Bayesian Half-normal Plot",
     cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5, xlim = c(0, max(u) + 0.1))
size <- c(rep(1.2, 12), rep(1.5, 3))
text(u, saeff, names(saeff), cex = size, col = c(rep(4, 12), rep(2, 3)))
```



3.2 HiGarrote Variable Selection

```
cat("HiGarrote Results:\n")
```

```
## HiGarrote Results:
```

```
print(HiGarrote(d, y))
```

```
## $nng_estimate
##         D         C       C:D
## 150.08473  53.21270 -39.13208
##
## $U
##    A  B  C  D A:B A:C A:D B:C B:D C:D
## 1 -1 -1 -1 -1   1   1   1   1   1   1
## 2 -1 -1  1  1   1  -1  -1  -1  -1   1
## 3 -1  1 -1  1  -1   1  -1  -1   1  -1
## 4 -1  1  1 -1  -1  -1   1   1  -1  -1
## 5  1 -1 -1  1  -1  -1   1   1  -1  -1
```

```
## 6  1 -1  1 -1  -1   1  -1  -1   1  -1
## 7  1  1 -1 -1   1  -1  -1  -1  -1   1
## 8  1  1  1  1   1   1   1   1   1   1
##
## $pred_info
## $pred_info$pred_model_matrix_info
## $pred_info$pred_model_matrix_info$uni_level
## $pred_info$pred_model_matrix_info$uni_level[[1]]
## [1] -1  1
##
## $pred_info$pred_model_matrix_info$uni_level[[2]]
## [1] -1  1
##
## $pred_info$pred_model_matrix_info$uni_level[[3]]
## [1] -1  1
##
## $pred_info$pred_model_matrix_info$uni_level[[4]]
## [1] -1  1
##
##
## $pred_info$pred_model_matrix_info$two_level_id
## [1] 1 2 3 4
##
## $pred_info$pred_model_matrix_info$quali_id
## NULL
##
## $pred_info$pred_model_matrix_info$quanti_id
## NULL
##
## $pred_info$pred_model_matrix_info$me_num
## [1] 1 1 1 1
##
## $pred_info$pred_model_matrix_info$contrast
## $pred_info$pred_model_matrix_info$contrast[[1]]
##      [,1]
## [1,]   -1
## [2,]    1
##
## $pred_info$pred_model_matrix_info$contrast[[2]]
##      [,1]
## [1,]   -1
## [2,]    1
##
## $pred_info$pred_model_matrix_info$contrast[[3]]
##      [,1]
## [1,]   -1
## [2,]    1
##
## $pred_info$pred_model_matrix_info$contrast[[4]]
##      [,1]
## [1,]   -1
## [2,]    1
##
##
```

```
## $pred_info$pred_model_matrix_info$model_type
## [1] 1
##
## $pred_info$pred_model_matrix_info$initial_colnames
## [1] "A" "B" "C" "D"
##
## $pred_info$pred_model_matrix_info$n
## [1] 8
##
## $pred_info$pred_model_matrix_info$quanti_D_norm
## $pred_info$pred_model_matrix_info$quanti_D_norm[[1]]
## [1] NA
##
## $pred_info$pred_model_matrix_info$quanti_D_norm[[2]]
## [1] NA
##
## $pred_info$pred_model_matrix_info$quanti_D_norm[[3]]
## [1] NA
##
## $pred_info$pred_model_matrix_info$quanti_D_norm[[4]]
## [1] NA
##
##
##
## $pred_info$beta_nng
##                [,1]
##  [1,]  0.000000e+00
##  [2,] -2.032597e-15
##  [3,]  2.493822e-01
##  [4,]  7.033741e-01
##  [5,] -5.621534e-19
##  [6,]  1.099785e-17
##  [7,]  1.575476e-30
##  [8,]  4.679359e-18
##  [9,] -1.491205e-15
## [10,] -1.833931e-01
##
## $pred_info$y_mean
## [1] 805
##
## $pred_info$y_sd
## [1] 213.3783
##
## $pred_info$lmod
##
## Call:
## lm(formula = form, data = data.frame(U, ori_y))
##
## Coefficients:
## (Intercept)            D            C          D:C
##         805          165           83          -73
##
##
##
```

```
## attr(,"class")
## [1] "HiGarrote"
```

4 Section 8.3: Designs with More than Two Levels

4.1 Orthogonal Arrays (L9, L18)

```
library(DoE.base)
library(SFDesign)
library(twinning)

full <- full.factorial(4, 3)

# L9 with two replicates
D <- undesign(oa.design(nfactors = 4, nlevels = 3, randomize = FALSE))
D <- data.matrix(D)
D1 <- rbind(D, D)
D1 <- (D1 - 1) / 2
mp1 <- maxpro.crit(D1, delta = 1/3^2) / 3^2
en1 <- energy(full, D1)

# L18
D <- undesign(oa.design(ID = L18, randomize = FALSE))
D <- data.matrix(D)
D2 <- D[, 2:5]
D2 <- (D2 - 1) / 2
mp2 <- maxpro.crit(D2, delta = 1/3^2) / 3^2
en2 <- energy(full, D2)

cat("L9 (2 reps): MaxPro =", round(mp1, 4), ", Energy =", round(en1, 4), "\n")
```

```
## L9 (2 reps): MaxPro = 0.5174 , Energy = 2.7684
```

```
cat("L18: MaxPro =", round(mp2, 4), ", Energy =", round(en2, 4), "\n")
```

```
## L18: MaxPro = 0.4271 , Energy = 2.7192
```

5 Section 8.4: Mixture Designs

5.1 Simplex-Lattice and Simplex-Centroid Designs

```
library(mixexp)
library(Ternary)

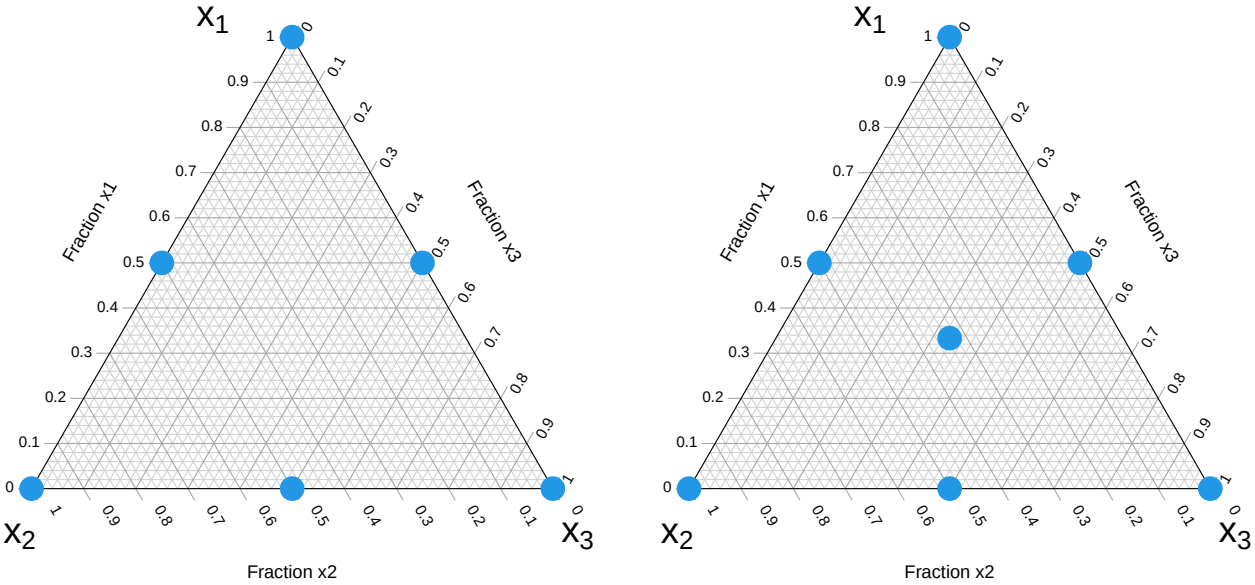
D1 <- SLD(3, 2)
D2 <- SCD(3)

par(mar = c(0.1, 0.1, 0.1, 0.1))
par(mfrow = c(1, 2))

TernaryPlot(axis.labels = seq(0, 1, by = 0.1),
            atip = expression(x[1]), btip = expression(x[3]), ctip = expression(x[2]),
            alab = "Fraction x1", blab = "Fraction x3", clab = "Fraction x2",
```

```
            lab.cex = 1, tip.cex = 2)
TernaryPoints(D1, cex = 3, col = 4, pch = 16)

TernaryPlot(axis.labels = seq(0, 1, by = 0.1),
            atip = expression(x[1]), btip = expression(x[3]), ctip = expression(x[2]),
            alab = "Fraction x1", blab = "Fraction x3", clab = "Fraction x2",
            lab.cex = 1, tip.cex = 2)
TernaryPoints(D2, cex = 3, col = 4, pch = 16)
```




```
par(mfrow = c(1, 1))
```

5.2 Support Points on Simplex

```
library(spacefillr)
library(support)

p <- 3
N <- 10000
u <- generate_sobol_set(N, p - 1)
x <- matrix(0, nrow = N, ncol = p)
x[, 1] <- (1 - u[, 1]^(1/2))
x[, 2] <- u[, 1]^(1/2) * (1 - u[, 2])
x[, 3] <- u[, 1]^(1/2) * u[, 2]

set.seed(1)
n <- 11
D1 <- sp(n, p, dist.samp = x)$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

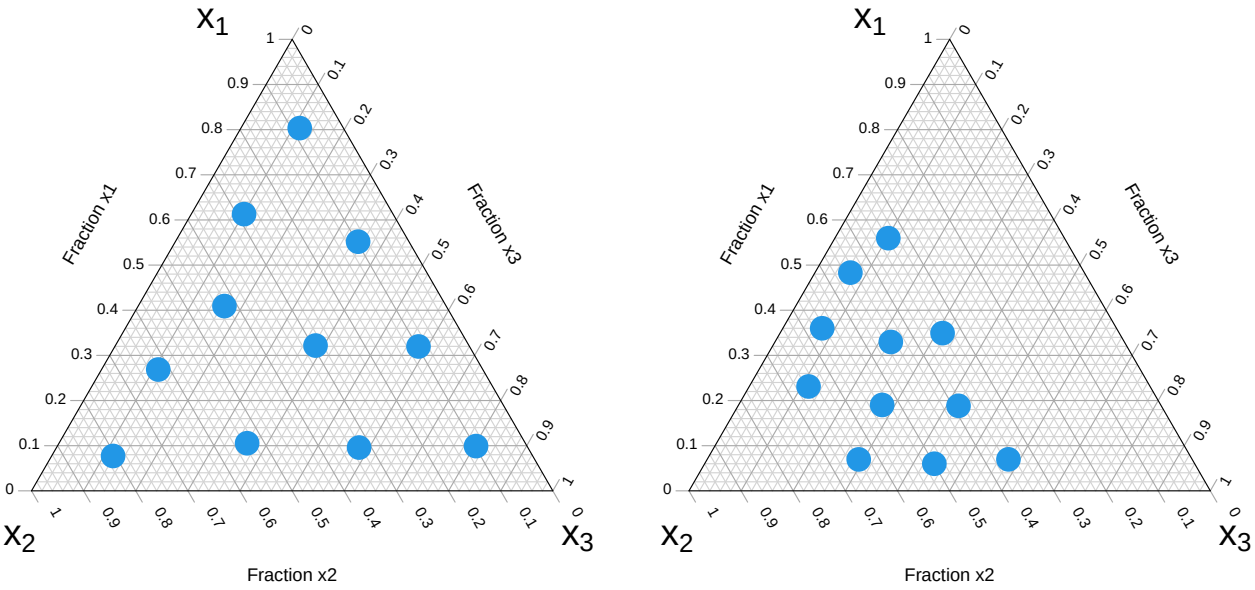
```
par(mar = c(0.1, 0.1, 0.1, 0.1))
par(mfrow = c(1, 2))
```

```r
# Full simplex
TernaryPlot(axis.labels = seq(0, 1, by = 0.1),
            atip = expression(x[1]), btip = expression(x[3]), ctip = expression(x[2]),
            alab = "Fraction x1", blab = "Fraction x3", clab = "Fraction x2",
            lab.cex = 1, tip.cex = 2)
TernaryPoints(D1, cex = 3, col = 4, pch = 16)

# Constrained region
CAND <- x
C1 <- (x[, 1] + x[, 2] < 0.7)
C2 <- (x[, 1] + x[, 2] > 0.3)
CAND <- CAND[C1 & C2, ]
D2 <- sp(n, p, dist.samp = CAND)$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

```r
TernaryPlot(axis.labels = seq(0, 1, by = 0.1),
            atip = expression(x[1]), btip = expression(x[3]), ctip = expression(x[2]),
            alab = "Fraction x1", blab = "Fraction x3", clab = "Fraction x2",
            lab.cex = 1, tip.cex = 2)
TernaryPoints(D2, cex = 3, col = 4, pch = 16)
```




```r
par(mfrow = c(1, 1))
```

6 Summary

This chapter covered:

1. **Two-level fractional factorial designs** with half-normal plots
2. **Bayesian-inspired designs** using Kriging priors and HiGarrote
3. **Orthogonal arrays** (L9, L18) for multi-level factors
4. **Mixture designs** including SLD, SCD, and support points on simplex